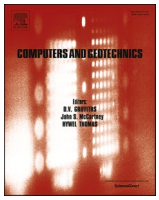
Computers and Geotechnics 174 (2024) 106657

Contents lists available at ScienceDirect

Computers and Geotechnics

journal homepage: www.elsevier.com/locate/compgeo



Research Paper

Parameters in play: AlphaZero-Inspired AI for autonomous parameter identification in soil constitutive and finite element models

Javad Ghorbani [a,1,*], Sougol Aghdasi [b], Majidreza Nazem [c], John S McCartney [d], Jaynatha Kodikara [a]

[a] ARC Smart Pavements Hub, Dept. of Civil Engineering, Monash University, VIC 3800, Australia
[b] School of Geography, Earth and Atmospheric Sciences, University of Melbourne, Melbourne, Victoria, Australia
[c] School of Engineering, Royal Melbourne Institute of Technology (RMIT), 124 La Trobe St, Melbourne, VIC 3000, Australia
[d] University of California, San Diego, La Jolla, CA 92093-0085, USA

ARTICLE INFO

Keywords:
Artificial intelligence
Constitutive modeling
Optimization
Geotechnical engineering
Self-learning methods

ABSTRACT

In geotechnical engineering, the precise identification of essential soil parameters from sensing and experimental data is vital for the accuracy of constitutive and finite element models. However, the complexity of sophisticated soil models often makes this task challenging. Traditional optimization methods that rely on gradient information often fall short in this class of problems, due to their struggle with black box models lacking clear gradient pathways. Gradient-free methods, though circumventing the need for direct gradient data, can still miss out on integrating previous insights when faced with new information. To tackle these issues, our study presents a cutting-edge method inspired by the mechanisms underlying AlphaZero, DeepMind's acclaimed algorithm that excels in mastering complex strategic games through autonomous learning. By adopting a comparable self-learning technique, our approach reinvents the task of parameter identification of advanced geotechnical models as a strategic game. It draws a parallel between optimizing model parameters and the complex task of developing victorious chess tactics. This method utilizes a blend of deep learning for initial estimations and Monte Carlo Tree Search (MCTS) for finer adjustments, promoting a self-enhancing calibration process. Such an approach paves the way for a more self-reliant and intelligent parameter identification methodology from sensing and experimental data. The outcomes of our study demonstrate the robustness and versatility of this approach across various geotechnical models, ranging from the parameter identification of sophisticated constitutive models to more complex applications involving inverse analyses using finite element models that include interactions between mechanical sensing devices and unsaturated soils.

1. Introduction

The field of soil mechanics stands on the brink of a profound transformation towards a more autonomous and intelligent paradigm (Yu et al., 2023, Li et al., 2024). At the forefront of this evolutionary shift lies the potential integration of sophisticated self-learning models, driven by artificial intelligence (AI), into geotechnical simulations. These models are envisaged to encapsulate and uphold the extensive wealth of empirical observations and theoretical advancements in soil mechanics, while also being engineered to adjust to new data inputs dynamically and efficiently. This capability has the potential to significantly enhance their practical utility and flexibility. This would represent a critical leap forward in the application of advanced and more accurate models in geotechnical design and analysis.

The automatic estimation of essential soil parameters (vital for the precision of advanced geotechnical models such as constitutive and finite element method models) from sensing or experimental data presents a unique challenge in computational mechanics. This challenge primarily stems from the nature of these models, which often operate as black-box computational tools or externally executable software. This characteristic significantly limits the use of gradient-based optimization algorithms. Although such algorithms are typically recognized for their speed and efficiency, they depend on accessible gradient and Hessian information to iteratively find the parameters. This information remains

* Corresponding author.
E-mail address: javad.ghorbani@rmit.edu.au (J. Ghorbani).
[1] Current affiliation: School of Engineering, Royal Melbourne Institute of Technology (RMIT), 124 La Trobe St, Melbourne, VIC, 3000, Australia.

https://doi.org/10.1016/j.compgeo.2024.106657
Received 14 May 2024; Received in revised form 10 July 2024; Accepted 4 August 2024
Available online 12 August 2024

elusive in black-box scenarios. Despite the recognition of some gradient-based methods in the literature (e.g., Zentar et al., 2001, Lecampion et al., 2002, Ledesma et al., 1996, Seidl and Granzow, 2022, Kadlíček et al., 2022), their applicability is generally limited in working with black-box models.

In response to these limitations, a number of researcher adopted gradient-free optimization methods, such as Particle Swarm Optimization (PSO) (Machaček et al., 2022), Genetic Algorithms (GA) (Levasseur et al., 2008, Mendez et al., 2021, Robson et al., 2024), and Differential Evolution (Machaček et al., 2022), among others. These methods have emerged as practical alternatives, offering a pathway to bypass the limitations associated with gradient-based approaches. However, it is important to note that their application has been primarily confined to the identification of parameters in constitutive models.

Although gradient-free methods like GAs and PSO provide robust alternatives to gradient-based techniques, their efficacy heavily depends on the diversity present within their populations or swarms which is crucial for avoiding premature convergence on suboptimal solutions. Maintaining this diversity, however, can be challenging, often requiring re-tuning or customization for new datasets or variations in models, particularly those that are complex or black-box in nature. Furthermore, a significant drawback of gradient-free methods such as GA is their inability to retain knowledge gained from previous optimizations across different initial conditions. Each new set of conditions or dataset can necessitate a fresh optimization process, as these algorithms lack mechanisms to leverage insights from past optimizations. The capability to retain learned experiences becomes particularly advantageous in applications such as digital twin systems or continuous ground monitoring (Ghorbani and Kodikara, 2024), which demand frequent recalibration due to evolving sensing data. Methods possessing this capability enable swift and automated recalibration. In contexts involving real-time data analysis, manually adjusting optimization settings with each new dataset or change in initial conditions is generally impractical.

To address these challenges, the present study draws inspiration from the pioneering principles embodied by AlphaZero, a program celebrated for its self-teaching prowess in mastering strategic games like chess and Go, as developed by DeepMind (Silver et al., 2017). AlphaZero's distinction lies in its ability to self-guided learning and enhancement, based on the game's fundamental rules. By integrating deep neural networks with an advanced tree search algorithm, AlphaZero can predict, evaluate, and progressively refine its strategies through insights gained from self-play and simulations of outcomes. This mechanism of self-optimization is the cornerstone of AlphaZero's capability to uncover innovative strategies and secure a position of dominance. This innovative approach offers valuable insights for developing potentially more adaptive and broadly applicable optimization methodologies in computational mechanics.

By developing an AlphaZero-inspired approach to the problem of parameter identification from sensing and experimental data, we aim to leverage AI's potential to overcome the limitations of both gradient-based and gradient-free optimization methods. The envisioned algorithm requires minimal human inputs and data during the process of self-learning and training. It adopts a game-like strategy for parameter identification from data, where model parameters are adjusted through a process of exploration, prediction, and refinement. The versatility and effectiveness of our approach are demonstrated across various models, from advanced constitutive models to a complex boundary value problem involving contact mechanics. The results collectively indicate its broad capability to deliver robust calibration outcomes in a diverse range of problems commonly encountered in numerical analysis of geotechnical problems.

First, we establish the foundational approach of this paper towards parameter identification problems, discussing key concepts through straightforward examples. Subsequently, we discuss more complex applications in geotechnical engineering, such as determining parameters for an advanced soil plasticity model from empirical data. Additionally, we explore parameter determination from sensing data, wherein the interaction between the mechanical sensing device and the unsaturated soil is modelled as contact mechanics-based boundary value problems within a fully coupled framework.

2. Solution procedure

AlphaZero developed by DeepMind, has shown impressive ability in learning and mastering games like chess, shogi, and Go from scratch, utilizing self-play without human intervention. Its methodology, blending deep neural networks with MCTS (Silver et al., 2017), provides a robust mechanism for predicting and evaluating strategies to secure a winning position.

Inspired by AlphaZero, the proposed approach for obtaining key soil parameters in geotechnical models from sensing and experimental data reimagines the process of parameter identification as a strategic game. Such a game aims to minimize the discrepancies between model predictions and observed data. In such a game the model parameters may be viewed as chess pieces, each contributing uniquely towards achieving the goal of the game.

2.1. Self-play

Fig. 1 presents a schematic and initial look into the 'game' of parameter identification where the key self-play mechanism is brought to the forefront. In this setup, two distinct roles emerge: the explorer and the learner agents. The explorer agent is tasked with adjusting a specific set of model parameters, while the learner focuses on learning from the outcome of these adjustments on the model's predictions.

The figure schematically shows a model described by M parameters, labeled from a_1 to a_M, which are to be determined through field trials or laboratory data. These parameters are initially assigned specific values, aggregated in set $\mathbf{A}$, which forms the baseline for the parameter identification process as follows:

$$\mathbf{A} = \{a_j \mid j = 1, 2, \cdots, M\} \tag{1}$$

The self-play process may include a total number of N_T trials in which these parameters may be modified within the set $\mathbf{G}$, which holds the initial conditions denoted by g. Consequently, the set $\mathbf{B}$, which captures all elements subject to alteration during self-play, can be broadly defined as:

$$\mathbf{B} = \{g_k, a_j\} \tag{2}$$

where $j = 1, 2, \cdots, M$ and $k = 1, 2, \cdots, N_{IC}$ with N_{IC} representing the number of parameters determining the initial conditions of the analysis.

The permissible adjustments on $\mathbf{A}$ are bounded by user-defined maximum and minimum values for each parameter. Specifically, the boundaries for these adjustments are delineated as:

$$\mathbf{A}_{max} = \{a_{j_{max}} | \, j = 1, 2, \cdots, M\} \tag{3}$$

and

$$\mathbf{A}_{min} = \{a_{j_{min}} | \, j = 1, 2, \cdots, M\} \tag{4}$$

where $\mathbf{A}_{max}$ and $\mathbf{A}_{min}$ respectively contains the maximum and minimum for the parameters involved in self-play.

Similarly, for the range of initial conditions we may define:

$$\mathbf{G}_{max} = \{g_{k_{max}} | k = 1, 2, \cdots, N_{IC}\} \tag{5}$$

and

$$\mathbf{G}_{min} = \{g_{k_{min}} | k = 1, 2, \cdots, N_{IC}\} \tag{6}$$

These parameters and their limits act as the 'rules of the game', directing

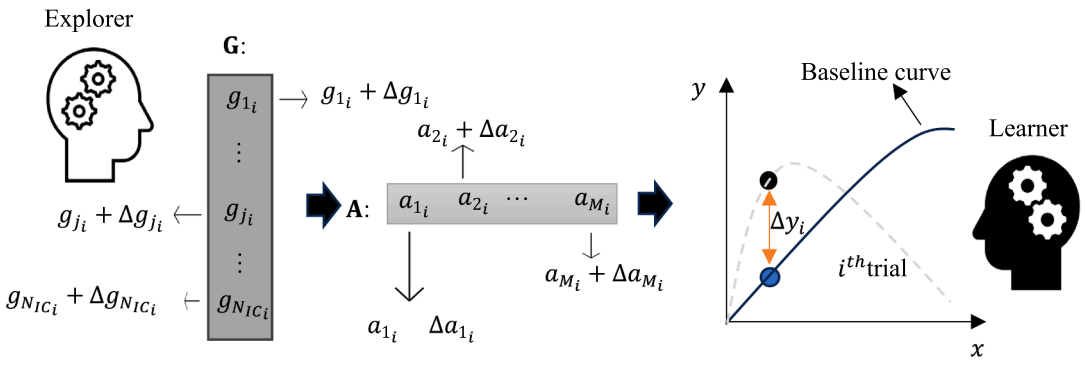


Fig. 1. Schematic overview of the self-play mechanism in calibration problems during i^{th} trial.

how parameters may be altered during the calibration process. The mechanism for parameter adjustment involves randomly selecting values from uniform distributions that are specifically defined within these bounded areas, for both the initial conditions and the calibration parameters themselves.

Upon implementing parameters adjustment, the learner agent then observes the deviations of the predictions (for example, from the curve resulting from the i^{th} trial shown in Fig. 1) compared to the baseline. Let's consider the schematic model shown in Fig. 1 is denoted by H, and incorporates N_{IC} numbers of initial conditions, the M parameters of interest, and an additional set of parameters not involved in the self-play mechanism represented as $\boldsymbol{\beta}_{ex}$. The deviation observed can be mathematically articulated as:

$$\Delta \mathrm{y}_i = H(x_i, \overline{\mathbf{G}}_i, \overline{\mathbf{A}}_i, \boldsymbol{\beta}_{ex}) - H(x_i, \overline{\mathbf{G}}_i, \mathbf{A}_i, \boldsymbol{\beta}_{ex}) \tag{7}$$

The equation essentially calculates the difference (Δy_i) between the model's predictions with adjusted parameters and the original (or baseline) parameters at a specific state. In this equation, $i = 1,2,\cdots,N_T$. Also, $\overline{\mathbf{G}}_i$ and $\overline{\mathbf{A}}_i$ respectively are the sets containing altered values of initial conditions and baseline parameters in the i^{th} trial.

For the schematic example shown in Fig. 1, the conclusion of the self-play process involves the learner agent's effort to train a surrogate deep learning model, denoted as f. This model is designed to process inputs consisting of the states $\mathscr{S}$, which encapsulate the state information. Within this framework, the state set for the i^{th} game or trial $\mathscr{S}$ for the schematic example shown in Fig. 1 may be defined as follows:

$$\mathscr{S} = \{(x_i, y_i) | i = 1, 2, \cdots, N_T\} \tag{8}$$

The input of the surrogate model also includes the difference between predictions derived from modified values and those obtained using baseline values, given by:

$$\mathscr{D} = \{\Delta y_i | i = 1, 2, \cdots, N_T\} \tag{9}$$

The applied adjustments to the base parameters are also shown by:

$$\Delta\mathbf{A} = \overline{\mathbf{A}} - \mathbf{A} = \left\{\Delta a_{j_i} \middle| \, j = 1, 2, \cdots, M \text{ and } i = 1, 2, \cdots, N_T\right\} \tag{10}$$

We also define matrices $\mathbf{X}$ and $\mathbf{Y}$ by:

$$\mathbf{X} = \{\mathscr{S}, \mathscr{D}\} \tag{11}$$

and

$$\mathbf{Y} = \Delta\mathbf{A} \tag{12}$$

Given the above definitions, the deep surrogate model can be expressed as:

$$f(\mathbf{X}) = \mathbf{Y} \tag{13}$$

2.2. Parameter identification

Upon completing the learner agent's task, the objective shifts to utilizing the learned surrogate deep model to deduce parameters based on a set of observed data. If we denote the total number of observations by N_o, the trained surrogate model steps in to predict N_o sets of adjustments. These predicted adjustments are compiled in $\Delta\widetilde{\mathbf{A}}$ as follows:

$$\Delta\widetilde{\mathbf{A}} = \left\{ \left(\Delta\widetilde{a}_{j_l} \middle| \, l = 1, 2, \cdots, N_o\right) \middle| \, j = 1, 2, \cdots, M \right\} \tag{14}$$

To distill these predictions into a cohesive strategy, we compute $\Delta\widetilde{\mathbf{A}}_{ave}$, the average across all proposed adjustments in $\Delta\widetilde{\mathbf{A}}$ as follows:

$$\Delta\widetilde{\mathbf{A}}_{ave} = \left\{\Delta\widetilde{a}_{ave_j} \middle| \, j = 1, 2, \cdots, M\right\} \tag{15}$$

where

$$\Delta\widetilde{a}_{ave_j} = \frac{1}{N_o}\sum_{l=1}^{N_o} \Delta\widetilde{a}_{j_l}, \forall j \in \{1, 2, \cdots, M\} \tag{16}$$

This average adjustment serves as the pivotal starting point for the MCTS algorithm, which is dedicated to refining these adjustments further.

2.2.1. Monte Carlo Tree Search

MCTS is a heuristic search algorithm used for making optimal decisions in each domain by taking random samples in the decision space and building a search tree based on the results. MCTS operates by iteratively constructing a search tree until a computational budget (e.g., time, memory, or iterations) is exhausted. Each iteration generally consists of four key steps: selection, expansion, simulation, and backpropagation. The schematic representation of these process is shown in Fig. 2 and involves the following items:

- **Nodes**: In the MCTS framework, each node symbolizes a specific state within the decision space. The initial node, often referred to as the root node, represents the current state from which the search commences. In the context of the problem schematically described in Fig. 1, the initial node is defined by the current adjustment to the parameters. As the search tree expands, each subsequent node embodies the result of a decision or action taken from the preceding state or samples drawn from a predefine set of actions N_{action}, a set of $\Delta\widetilde{\mathbf{A}}_{action}$ that is shown by:

$$\Delta\widetilde{\mathbf{A}}_{action} = \left\{\Delta\widetilde{a}_{ave_j} + \left(\mathscr{P}_j\right)_h \middle| \, j = 1, 2, \cdots, M \text{ and } h = 1, 2, \cdots, N_{action}\right\} \tag{17}$$

where $\left(\mathscr{P}_j\right)_h$ is the set containing the h^{th} alteration applied to j^{th} average adjustment value $\Delta\widetilde{a}_{ave_j}$, aiming to explore possible improvements by modifying the average parameter values. These alterations applied to $\Delta\widetilde{a}_{ave_j}$ are derived from a uniform distribution obtained

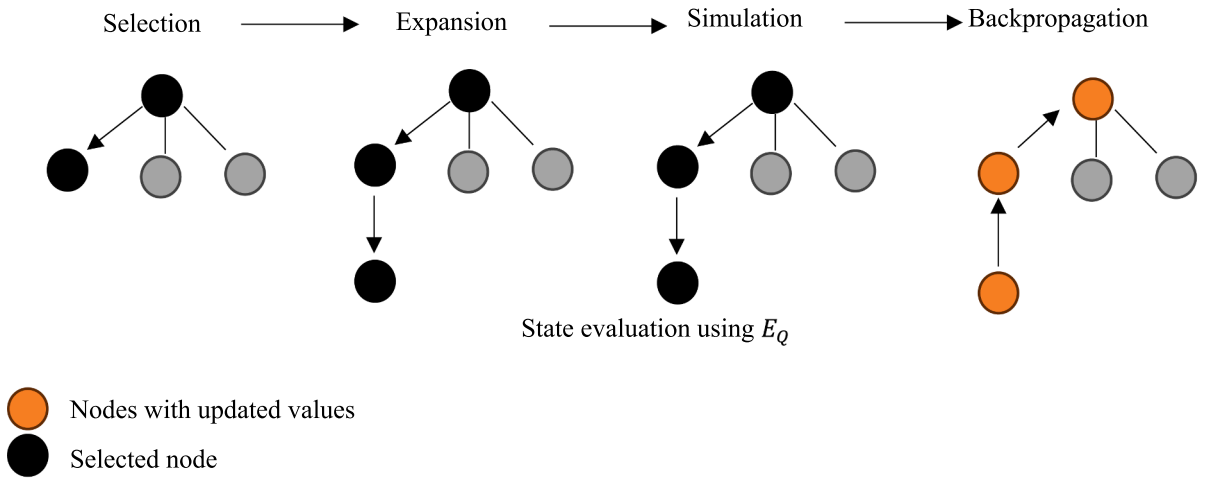


Fig. 2. Schematic overview of four stages of MCTS.

according to:

$$\mathscr{P} = \{\mathrm{UN}(lb_j, ub_j) \mid j = 1, 2, \cdots, M\} \tag{18}$$

where UN is a function generating from a uniform distribution, with the terms lb_j and ub_j denoting the lower bound and upper bound of the distribution.

We have defined the bounds for alterations applied to $\Delta\widetilde{a}_{ave_j}$ as a multiplier of the span of all conceivable adjustments for each parameter j where we defined these bounds as follows:

$$(lb_j, ub_j) = \left(n_{sc}\left(\Delta\widetilde{a}_{min_j} - \Delta\widetilde{a}_{ave_j}\right), n_{sc}\left(\Delta\widetilde{a}_{max_j} - \Delta\widetilde{a}_{ave_j}\right)\right)\forall j \in \{1, 2, \cdots, M\} \tag{19}$$

where $\Delta\widetilde{a}_{min_j}$ and $\Delta\widetilde{a}_{max_j}$ represent the maximum and minimum of parameter j in the series of predictions stored in $\Delta\widetilde{\mathbf{A}}$ in Equation (14).

The value of n_{sc} can generally depend on the assumed parameter range's accuracy relative to the best solution. Nonetheless, our experimental work with the presented examples indicates that a multiplier value (n_{sc}) in the range of 2 to 3 should generally be appropriate for n_{sc}. For the subsequent examples, we have set n_{sc} to 2 as the default value.

- **Selection**: In this initial phase, the algorithm traverses the existing tree structure from the root to a leaf node. Each node encountered along this path signifies a decision point where a specific parameter adjustment has been applied. Guided by a traversal policy like the Upper Confidence Bound (UCB), the algorithm navigates to nodes that strike a balance between exploring untested parameter combinations and exploiting combinations known to yield closer fits to observed data.

The UCB algorithm was introduced as part of a family of solutions to the multi-armed bandit problem, where a gambler must decide which arm of a bandit (slot machine) to pull to maximize their total reward over several plays. The UCB algorithm (Auer et al., 2002), one of the most commonly used index in MCTS, is defined by the equation:

$$UCB = R_h + \sqrt{\frac{\ln n}{n_h}} \tag{20}$$

where:

- R_h is the average reward obtained from action h. The term directs the selection towards actions that have historically yielded higher rewards.
- n is the total number of times all actions have been selected and n_h is the number of times action h has been selected. The second term $\sqrt{\frac{\ln n}{n_h}}$ generally encourages exploration of less-selected actions.

For state evaluation, let us consider a set $\mathbf{Q} = \{Q_p | p = 1, 2, \cdots, N_Q\}$ containing N_Q different quantities of interest. These quantities serve as the metrics against which the model's performance is assessed in comparison to observed data. We define an adjusted measure of relative error, E_Q as follows:

$$E_Q = \sum_{p=1}^{N_Q}\sum_{l=1}^{N_o}\frac{\left|Q_{predicted_{pl}} - Q_{observed_{pl}}\right|}{1 + \left|Q_{observed_{pl}}\right|} \tag{21}$$

where $l = 1, 2, \cdots, N_o$, and $Q_{observed_{pl}}$ and $Q_{predicted_{pl}}$ are the predicted and observed values of of the p^{th} quantity of interest for the l^{th} observation, respectively.

The adoption of a relative error-like metric over traditional measures like Mean Absolute Error (MAE) is primarily motivated by the inherent challenges posed by datasets that feature quantities of interest with significant variations in magnitude. This situation is notably prevalent in disciplines such as soil testing.

The adjustment comes from the denominator, $1 + \left|Q_{observed_{pl}}\right|$, where the addition of a unit to $\left|Q_{observed_{pl}}\right|$, ensures that the relative error does not become excessively large for very small values of $Q_{observed}$, a common issue with the standard relative error formula. In this context, we define $R_h = -E_Q$, meaning that minimizing the error maximizes the reward.

- **Expansion**: Upon reaching a leaf node during the selection phase that has not been fully explored, the expansion phase adds a new node to the tree.

In the context of parameter identification problems, this entails selecting a new set of alterations from the defined action space for exploration. This new node represents a hypothetical model state with different parameter values, thereby expanding the search space of potential solutions.

- **Simulation**: A simulation is executed from the newly added node to evaluate the state. For the problems of parameter identification, the simulation phase involves simulating the model's behavior with the updated parameter set and comparing the resulting predictions to observed data. The outcome of this simulation offers insights into the effectiveness of the parameter adjustments represented by the node.

The primary criterion for evaluating a node is the extent to which it minimizes the discrepancy between the model's predictions and the observations. This discrepancy is quantified by the adjusted measure of relative error (E_Q) between the predicted and all the observed values.

- **Backpropagation**: Results obtained from the simulation phase are then propagated back up the tree, updating each ancestor node with fresh information regarding the value (i.e., the potential success in minimizing the deviation from the observations in terms of the adjusted measure of relative error) of the parameter adjustments it represents. This iterative update process enables the algorithm to refine its understanding of which paths in the tree (i.e., sequences of parameter adjustments) are most promising for achieving the calibration objective.

A Python code is developed to implement the aforementioned algorithms. The self-play process (Algorithm 1) utilizes external Python libraries such as NumPy and Pandas, while Torch and Joblib are used for training and saving the deep surrogate model.

Algorithm 1 Self-learning and training procedures.

- Initialization for self-learning
  - Set base parameters, parameter ranges, N_T, and the template input file for external processor (if applicable).
- Generate Data:
  - Set $i = 0$.
  - Initiate empty lists called **`param_modifications`** and **`change_in_predictions.`**
  - While $i < N_T$:
    - Calculate the model prediction based on the base parameters and store the results as base predictions.
    - Draw random values from the determined bounds, make the input file for external processor based on the modified parameter (if applicable).
    - Calculate the difference between the model prediction and the base predictions.
    - Append the modifications to the parameters relative to the base parameters to **`param_modifications`** and the difference between the model prediction and the base predictions to **`change_in_predictions.`**
    - i++
- Training Phase:
  - Set the deep surrogate model structure, validation/training ratio as well as the regularization setting.
  - Execute training of the deep surrogate model based on the data stored in **`param_modifications`** and **`change_in_predictions.`**
  - Save the trained model.

Algorithm 2. Definitions of Node and Monte Carlo Tree Search Classes.

- **Initialization and input**
  - **Input**: Deep surrogate model, action space, number of parameters, file paths, including input, output, and the executable files, for external processes (if applicable).
- **MCTS Node Class Definition**
  - **Attributes**:
    - `state`: Current parameter set.
    - `parent`: Parent node.
    - `action`: Action leading to this node.
    - `children`: List of child nodes.
    - `visits`: Count of node visits.
    - `total_value`: Cumulative reward of the node.
  - **Methods**:
    - `is_leaf()`: Returns `True` if the node has no children.
    - `select_child()`: Selects the best child node based on the UCB formula and Algorithm 4.
    - `expand(action_space)`: Creates child nodes for each action in the action space. This involves:
      1) Creating a new state from the action space by calling the `apply_action` method of Monte Carlo Tree Search Class.
      2) Create a new node using the new state.
      3) Append the new node as a child node.
    - `update(value)`: Updates the visit count and total value with the given value.
    - `best_action()`: Determines the best action from this node based on E_Q.

(*continued on next column*)

(*continued*)

- **Monte Carlo Tree Search Class Definition**
  - **Attributes**:
    - `action_space`: Set of N_{action} possible actions (parameter adjustments).
    - `num_params`: Number of parameters in the model.
    - `file_paths`: Paths for handling external processes.
  - **Methods**:
    - `simulate(node)`: Evaluates the node's state by running a simulation. This involves:
      1) Modifying the input file (if applicable) based on the node state.
      2) Call the executable software and run the simulation using the modified input file.
      3) Compute E_Q following Equation (21).
    - `search(initial_state, num_simulations)`: Executes MCTS to identify the best action for the initial state following Algorithm 3.
    - `apply_action(state_dict, action)`: Applies the given action to the state, updating the parameters.
    - `action_leading_to(child_node)`: Retrieves the action leading to the specified child node.

A custom MCTS is developed and detailed in Algorithm 2. The MCTS method comprises two main classes: Node and Monte Carlo Tree Search. These classes, along with their attributes and methods, are described in Algorithm 2. Additionally, a key component of the MCTS method is the search algorithm which iteratively finds the best possible state, as outlined in Algorithm 3. A key sub-algorithm in the MCTS method for selecting the best child node is presented in Algorithm 4.

Algorithm 3 Search algorithm for finding the best action.

- **Initialization**:
  - Create the root node of the MCTS tree with the initial state.
- **Simulation Loop**:
  - Repeat the following steps for a total of N_{MCTS} times:
    1 **Node Selection**:
      - Start from the root node.
      - Traverse the tree by selecting the best child node at each step until a leaf node is reached.
        - A node is a leaf if it has no children.
        - Use the `select_child()` method to choose the best child.
    - **Node Expansion**:
      2 If the leaf node has been visited before, expand the node by generating its children.
      - Use the `expand(action_space)` method to create child nodes for each action in the action space.
    - **Simulation**:
      3 Use the `simulate(node)` method to run the simulation and obtain the value.
    4 **Backpropagation**:
      - Backpropagate the simulation value up the tree to update the visit counts and total values of the nodes.
        - Start from the current node and move up to the root, updating each node.
        - Use the `update(value)` method to update the node's statistics.
- **Return Best Action**:
  - After completing all simulations, determine the best action from the root node.
  - Use the `best_action()` method to identify the action leading to the highest average value.
  - Return the best action and the negative value of the best simulation result.

Algorithm 4. Selecting the best child node.

- **Initialization**:
  - Initialize `best_score` to the negative value of a very large number.
  - Initialize `best_child` to `None`.
- **Iterate Through Children**:
  - For each child node in the current node's children:
    1) **Check Visits**:
      - If the child node has not been visited:
        - Set Score to a very large number (positive) to prioritize exploration of this node.
      - Else:

(*continued on next page*)

(*continued*)

- Calculate the exploitation term in Equation (20) as the average value of the child node.
- Calculate the exploration term in Equation (20).
- Set
  Score = exploitation term + exploration term

2) **Update Best Score**:
- If the calculated Score is higher than `best_score`:
  - Update `best_score` to the current score.
  - Update `best_child` to the current child node.
  - **Return** `best_child`

2.3. Demonstrating algorithmic principles with a synthetic case study

In the following series of examples, we explain the fundamental operational dynamics of the designed algorithms. Specifically, we discuss the designed system through the lens of fitting a fourth-order polynomial to a set of artificially generated noisy observations. The polynomial in question is represented as

$$P(x) = a_5x^4 + a_4x^3 + a_3x^2 + a_2x + a_1 \tag{22}$$

where the number of parameters M is 5 in this example. The domain of interest for x is set within the interval $x \in [-1, 1]$.

The game rule is defined as follows:

$$a_j \in [-10, 10],\ j = 1, 2, \cdots, 5 \tag{23}$$

Also, the base parameters are defined as follows:

$$\mathbf{A} = \{1, 1, 1, 1, 1\} \tag{24}$$

Given the context of this example, the need for **G** is bypassed. Consequently, the parameters subject to adjustment, encapsulated within **B**, are directly aligned with the base parameters:

$$\mathbf{B} = \{1, 1, 1, 1, 1\} \tag{25}$$

The investigation proceeds by applying 10,000 simulations with random adjustments applied to the elements of **B**, during which both the state values and their deviations from the baseline predictions are recorded.

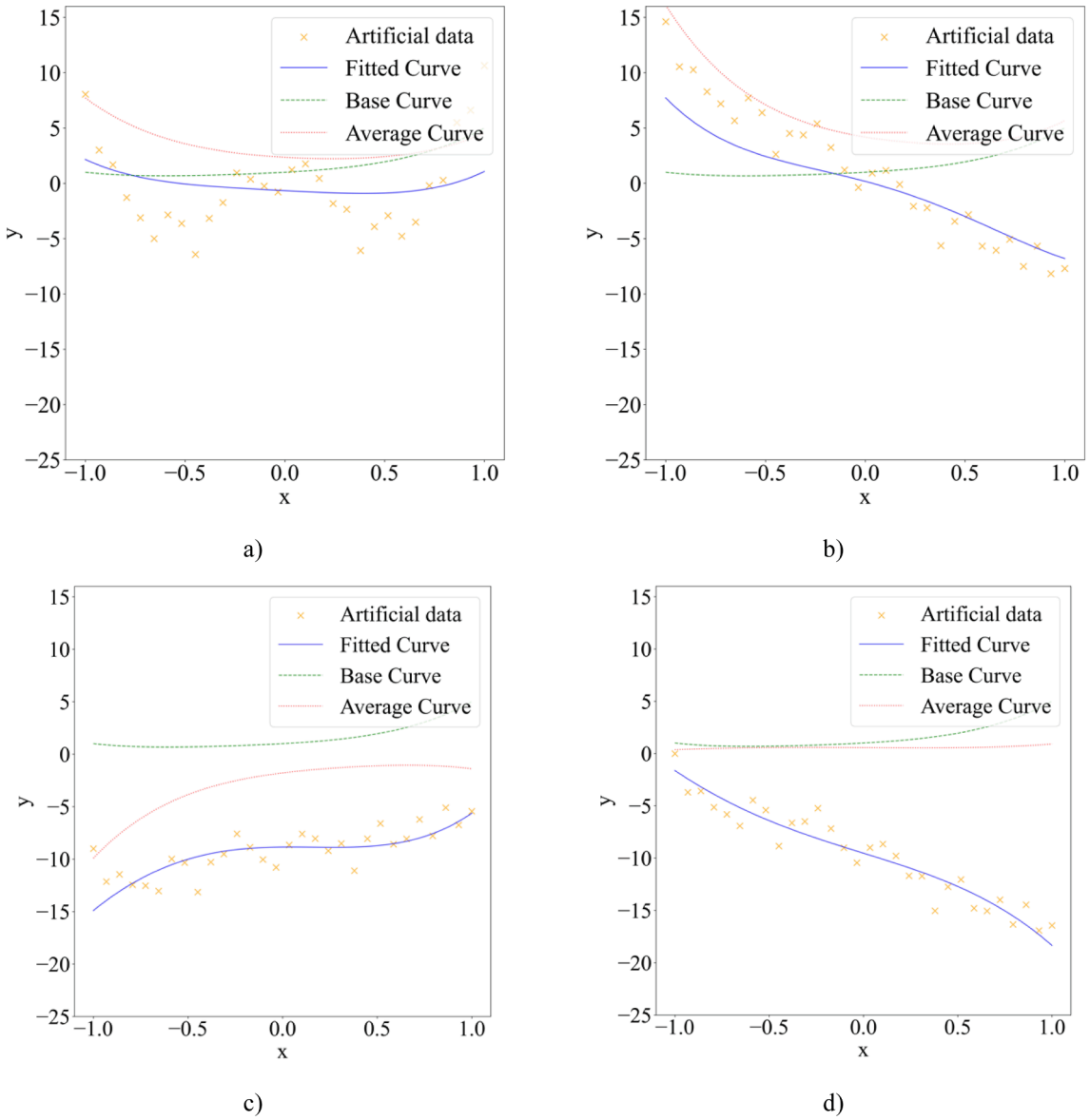


Fig. 3. Algorithm's performance in four different cases. Panels a) to d) show Cases 1 to 4, respectively. The labels "Fitted Curve", "Base Curve", and "Average Curve" denote predictions from the MCTS algorithm, outcomes based on base parameters, and results derived via ANN, respectively.

The collected data serves as the foundation for training an Artificial Neural Network (ANN). In working with ANN, it is crucial to minimize overfitting, which typically occurs when a model is too complex for the available dataset. To address this problem, we employed two strategies: starting with a simpler structure and regularization.

We began with a simpler model structure and gradually increased its complexity, using an 80/20 training-validation split to monitor performance. The ANN, designed with three input neurons to represent the variables stored in **X**, viz., (x, y and Δy), features two hidden layers each consisting of 64, and 32 neurons, respectively. Additionally, we incorporated regularization terms in our loss function to control and penalize excessive complexity. This technique discourages the model from becoming overly complex by imposing a penalty on the magnitude of the ANN weights.

The model also includes an output layer with five neurons corresponding to the polynomial's coefficients. To minimize overfitting in training the ANN mode and improve the model's generalization to unseen data, in all the presented examples, we adopt L2 regularization by integrating it into the optimizer, which helps to penalize large weights and reduce model complexity.

Following the training phase, the ANN model is deployed and the average of adjustments $\Delta\widetilde{\mathbf{A}}_{ave}$ is calculated and the action space is formed according to the approach outlined in Equation (18).

The scope of the search within the MCTS framework is governed by a modifiable parameter, N_{MCTS}, which has been assigned a value of 300 for the benchmark scenarios depicted in Fig. 3. This parameter specifies the total number of iterations (or moves) the algorithm will execute, starting from the root node that depicts the present state, marking the conclusion of the game. The sensitivity of the outcomes to variations in N_{MCTS} will be demonstrated and discussed in the following section, highlighting its impact on the efficiency and effectiveness of the search process. The state evaluation is performed by the adjusted measure of relative error introduced in Equation (21) by selecting the predicted y as the quantity of interest based on which the model's performance is assessed.

The relationship between N_{MCTS} and the size of the action space, N_{action}, is crucial for striking a balance between computational efficiency and the effectiveness of the algorithm. A larger action space often enhances accuracy, but achieving this improvement necessitates more thorough search efforts, which can significantly increase computational time and may not always be practical. In the benchmark cases examined in this study, a baseline value of 300 for N_{action} is used. Further analysis that will be presented in the subsequent section will explore how the model's performance is sensitive to variations in these parameters.

Fig. 3 illustrates the dynamics of the model's performance across four synthetically generated datasets. Each panel within the figure displays three unique curves: one representing the baseline established by the initial parameters, another derived from the outcomes following the training of the ANN model and the calculation $\overline{\mathbf{A}}$, and the third curve predicted by the MCTS process.

In every instance, MCTS effectively reduced the adjusted relative error in the previous ANN step, as detailed in Table 1.

The extent of this enhancement is generally influenced by factors including the specific rules of the game being simulated, the allowable ranges for the parameters, the complexity inherent in the dataset, and the depth of the training process. However, the distinguishing feature of this approach lies in its synergistic integration of deep learning with a sophisticated refinement process through MCTS. This dual-phase methodology demonstrates that, even in scenarios where the initial deep learning model might not achieve optimal performance, the algorithm retains the capacity to further refine and improve results, leveraging its strategic exploration and exploitation mechanisms.

The simulation process was executed on a MacBook Pro equipped with an Apple M2 Pro chip and 16 GB of memory, running macOS Sonoma version 14.1. This operation required approximately 0.05 s. Subsequent phases, including self-play and the training of the ANN model over 50 epochs, took approximately 0.52 s and 6.02 s, respectively.

Fig. 4 illustrates the effects of varying the MCTS breadth parameter (N_{MCTS}) and the action space size (N_{act}) on E_Q and computation time across the four distinct scenarios outlined in outlined in Fig. 3. Panels a) and b) analyze the case where $N_{act} = 100$, demonstrating how changes in N_{MCTS}/N_{act} influence the outcomes. Panels c) and d) extend this analysis to a larger action space size of $N_{act} = 300$, again observing the effects of varying N_{MCTS}/N_{act}. Finally, panels e) and f) demonstrate scenarios with an even larger action space ($N_{act} = 500$), maintaining consistent adjustments in N_{MCTS}/N_{act} (0.1, 0.2, 0.5, 1.0, 1.5, 2.0) to examine its impact under these conditions.

The graphs collectively show that computational time has a direct relationship with N_{MCTS} and N_{act}, indicating that as the breadth of MCTS and the size of the action space expand, so does the computational effort required for the calibration process.

Moreover, the analysis reveals a potential optimum for the adjusted relative error within a ratio range of approximately 0.5 to 1.0. Below this threshold, the scarcity of searches can restrict the algorithm's ability to discover effective solutions, limiting its overall effectiveness. On the other hand, ratios above this range might inadvertently foster calibration inefficiencies by prompting over-exploration. This results in excess computational time being allocated to the examination of less optimal paths within the decision tree.

A pivotal aspect to consider is the algorithm's iterative approach to refining node values based on new simulation data, which, while beneficial, poses the risk of diluting the impact of prior valuable insights when exposed to over-exploration. An excessive focus on exploring new, yet less promising paths can diminish the relevance of previously identified optimal actions. This phenomenon could lead to the algorithm overlooking or undervaluing these key actions due to their diminished prominence against a backdrop of newer, albeit less effective, data.

These observations highlight the critical need for a judicious balance between the breadth of the search and the available computational resources, to ensure an efficient calibration process. In the upcoming simulations, we will use $\frac{N_{MCTS}}{N_{act}}$ ratio of 1, unless stated otherwise.

3. Results and discussion

This section illustrates various potential applications of the proposed method within the field of geomechanics. We explore a spectrum of problems, ranging from analytical modeling to complex inverse boundary value problems that require finite element analysis.

3.1. Soil Water Retention Curves

Soil Water Retention Curves (SWRCs) are critical functions that delineate the relationship between soil saturation and suction for a given drainage path. This relationship is crucial in the mechanics of

Table 1
Summary of parameters and relative error metrics across various test scenarios.

Cases	a_5	a_4	a_3	a_2	a_1	Time (s)	E_Q after ANN	Final E_Q
Case 1	1.98	0.37	0.29	−0.91	−0.66	0.22	26.51	18.87
Case 2	2.81	−2.44	−2.53	−4.82	0.18	0.22	33.82	10.72
Case 3	0.87	4.46	−2.28	0.18	−8.86	0.25	18.9	3.49
Case 4	−0.44	−2.76	−0.02	−5.6	−9.53	0.23	27.96	5.31

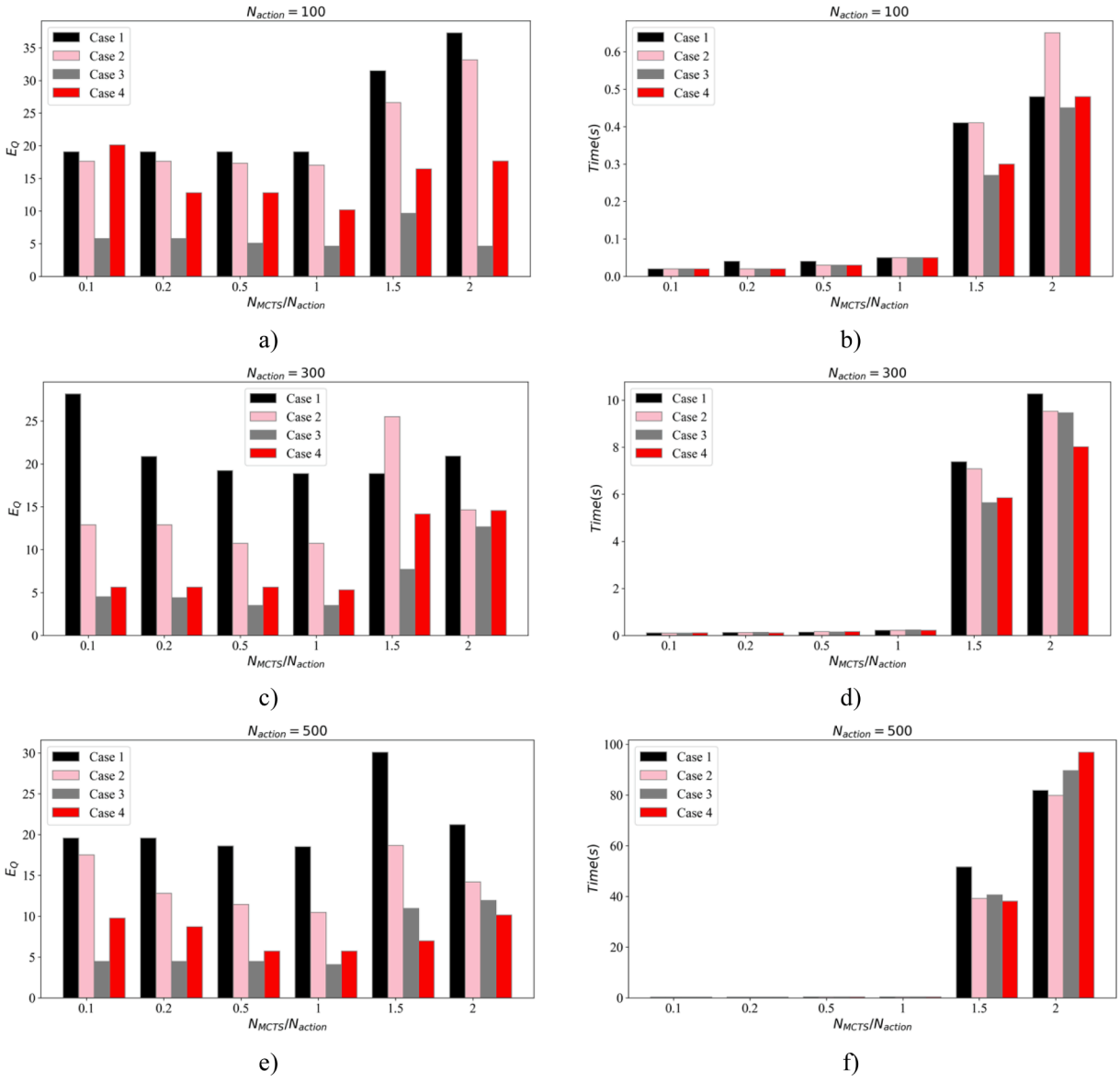


Fig. 4. The outcomes of a sensitivity analysis conducted on four distinct cases, varying N_{MCTS} and N_{action}. Panels a) shows the adjusted measure of relative error (E_Q) while panel b) focuses on the computational time required for the analyses when $N_{action} = 100$, adjusting the ratio N_{MCTS}/N_{action} from 0.1 to 2.0. Panels c) and d) replicate these examinations for $N_{action} = 300$ and panels e) and f) extend the analysis to $N_{action} = 500$, maintaining consistent N_{MCTS}/N_{action} ratios across scenarios.

unsaturated soils (Ghorbani and Airey, 2019) and is notably sensitive to volumetric changes (Kinikles et al., 2024, Ghorbani and Kodikara, 2023, Gallipoli et al., 2003a). Such sensitivity creates a complex three-way interdependence among saturation degree, suction, and void ratio with noted hysteresis. This often necessitates a trial-and-error approach for calibration.

In this section, we discuss the application of automatic calibration to this class of models, examine the impact of training quality on sensitivity. A SWRC equation suggested by Gallipoli et al., (2003b) is formulated by defining a scaled suction $p_c^* = p_c e^{\Omega'}$ where p_c is suction and Ω' is a material parameter that controls how the void ratio influences the SWRC.

$$S_w = \left(1 + \left(P^d p_c^*\right)^{n_v^d}\right)^{-m_v^d} \tag{26}$$

where S_w is the degree of saturation, P^d denotes the inverse of the air-entry value. n_v^d and m_v^d are the model parameters that control the slope of the SWRC. The experimental data in this section are from a silty sand provided by Maghvan et al., (2019).

For this problem, the set containing the unknowns is defined as follows:

$$\mathbf{A} = \left\{P^d, n_v^d, m_v^d, \Omega'\right\} \tag{27}$$

Similar to the previous illustrative examples, the need for **G** is bypassed. Consequently, the parameters subject to adjustment, encapsulated within **B**, are directly aligned with the base parameters and is defined as follows:

$$\mathbf{B} = \{1, 1, 1, 1\} \tag{28}$$

Variables x and y extend beyond dimensions discussed in the illustrative examples in Section 2. In this example, x represents suction and void ratio data and y may encompass the degree of saturation. Given this assumption, $\mathscr{I}$ may be defined as follows:

$$\mathscr{I} = \left\{\left(p_{ci}, e_i, S_{wi}\right) \middle| i = 1, 2, \cdots, N_T\right\} \tag{29}$$

and $\mathscr{D}$ is defined as follows:

$$\mathscr{D} = \{\Delta S_{wi} | i = 1, 2, \cdots, N_T\} \tag{30}$$

The empirical training ranges for these parameters are defined as:

$$P^d \in [0.01, 100],\ n_v^d \in [1.1, 10],\ m_v^d \in [0.1, 0.9],\ \text{and}\ \Omega^{'} \in [1, 10] \tag{31}$$

The domain of interest based on experimental data includes suction ranging from 0 to 100 kPa and void ratio changes between 0.5 and 1.0. The ANN model's architecture has been revised to include an input layer of four neurons reflecting the dimensionality of **X**, followed by four hidden layers each with 64 neurons, and an output layer of four neurons to represent the dimensionality of **Y**.

We first investigate the impact of training quality on the predictions by varying the number of trials N_T (5000, 25000, 40,000 and 50,000 trials) while maintaining $N_{act} = 2000$ and other analysis parameters the same in all the simulations.

Results are summarized in Table 2 and predictions are visually represented in Fig. 5. The table shows that increasing the number of trials from 5,000 to 25,000 results in a significant enhancement of the predictions' accuracy, as evidenced by a substantial reduction in relative error metrics. This observation aligns with a common machine learning principle: more data typically improves performance by enabling more comprehensive learning. However, when N_T is set to 40,000 trials, the improvements in prediction accuracy become less significant despite an increase in training time. This pattern of diminishing benefits, evident while the ANN model's structure remains unchanged, suggests that simply increasing the number of trials does not guarantee improved performance beyond a certain point. This occurs because additional data can introduce greater complexity, requiring modifications to the model's structure to effectively manage and utilize this new information. To achieve the best results, it may be prudent to strike a balance between the volume of data and model complexity, adjusting the architecture as needed to optimize performance across different datasets.

3.2. Elastoplastic Constitutive Model

Calibrating elastoplastic constitutive models represents a notably more complex endeavor compared to the examples previously discussed, primarily due to several key factors:

- Unlike the discussed analytical equations, constitutive modelling of geomaterials usually necessitates an incremental approach between stress and strain. Such incremental relationships require a stress integration scheme to approximate solutions incrementally. Therefore, these algorithms are generally slower than analytical models.
- Moreover, constitutive models and the associated stress integration schemes are frequently encapsulated within external codes or treated as black-box functions. This encapsulation necessitates a calibration approach that can interface effectively with external programs, often without direct insight into and influence on the internal structure of the models.
- Furthermore, constitutive model calibration often embodies a multi-objective optimization challenge. It is common to have multiple datasets under varying initial conditions and tests, with the overarching goal of achieving an overall good fit across all test datasets.

Table 2
Summary of parameters and relative error metrics in the SWRC calibration problem across various test scenarios.

	P^d(kPa^{-1})	n_v^d	m_v^d	Ω'	Self-play time (s)	E_Q
N_T = 5000	17.81	2.27	0.42	5.39	4.09	15.57
N_T = 25000	10.91	2.22	0.2	8.74	15.54	5.02
N_T = 40000	0.69	2.21	0.67	4.91	25.1	1.27
N_T = 50000	0.70	2.09	0.52	5.04	30.42	1.11

The selected constitutive model is developed by the first author and his colleagues Ghorbani and Airey (2021) and Ghorbani et al. (2021a) originally for unsaturated soils. In the special case of saturation (which is the focus of this example), the model shares many features with the SANISAND model by Dafalias and Manzari (2004), which has inspired several advanced models for granular soils (e.g., Chen et al., 2024, Petalas et al., 2020). The constitutive equations of the model are summarised in Table 3 and the shared features include the shear modulus, dilatancy, and kinematic hardening equations. The integration scheme and an associated FORTRAN-based implementation of this algorithm builds upon the foundational work by Sloan et al. (2001) extending its application into the domain of bounding surface plasticity as detailed by Ghorbani et al. (2021a) and Ghorbani and Airey (2021). This refined approach employs a comparative error analysis between a second-order accurate modified Euler solution and a first-order accurate Euler solution. This error comparison aims to devise an adaptive and automatic substepping scheme to enhance the integration scheme's efficiency. The integration tolerance was set to 1×10^{-5} in all the analyses.

The details of the constitutive model and the integration scheme developed by the first author and his colleagues is not repeated in this paper for brevity. The inputs to the stress integration algorithm are analysis conditions, material parameters, and strain increments, and the algorithm's outputs are stress values. This solution framework is depicted in an updated schematic in Fig. 6, which includes an intermediate step where the external process is invoked, and the produced results are leveraged for self-learning by the learner agent.

The model's parameters are also presented in this table. These parameters are organized into categories corresponding to elasticity, the critical state, kinematic hardening, and dilatancy.

Elasticity: The model employs two hypoelastic laws to describe the elastic behavior through bulk and shear moduli, characterized by K_0 and G_0 as the primary elasticity parameters.

Critical State: This aspect involves defining the critical state line in two primary spaces:

- In the void ratio (e) – mean effective stress (p') space, the model requires parameters N_c, λ, and α_{CSL}. Here, N_c denotes the intersection of the critical state line with the void ratio axis at unit mean effective stress, λ is the slope of this line, and α_{CSL} controls its curvature (Ghorbani et al., 2021a).
- In the deviatoric stress (q) – mean effective stress (p') space, the model needs an interpolation function $g(\theta_l, \alpha')$, that relies on the Lode angle (θ_l) to estimate the critical state stress ratio. The parameter α' is the ratio between the slope of the critical state line in the $q - p'$ space obtained in triaxial extension (M_e) to its counterpart in triaxial compression (M_c).

Kinematic Hardening and Dilatancy: These mechanisms are governed by seven parameters: n^b, h_0, c_h, n^d, A_0, c_z, and z_{max}. Except for c_z and z_{max}, which are specifically relevant to cyclic loading conditions, as detailed by Dafalias and Taiebat (2016) , and are consequently omitted from the calibration process for scenarios focusing on monotonic loading, the remaining parameters are integral to defining the soil's response under monotonic loading conditions.

In the context of constitutive model calibration, the variables x and y depicted in Fig. 1 extend beyond singular dimensions discussed in the illustrative examples in Section 2. Here, x could generally represent axial or deviatoric strain, metrics routinely measured during triaxial testing. Conversely, y could encompass a variety of outputs such as deviatoric stress (q), mean effective stress (p'), void ratio, or volumetric strain. Consequently, the model might need to interpret multiple components and curves simultaneously, introducing a layer of complexity to the learning process.

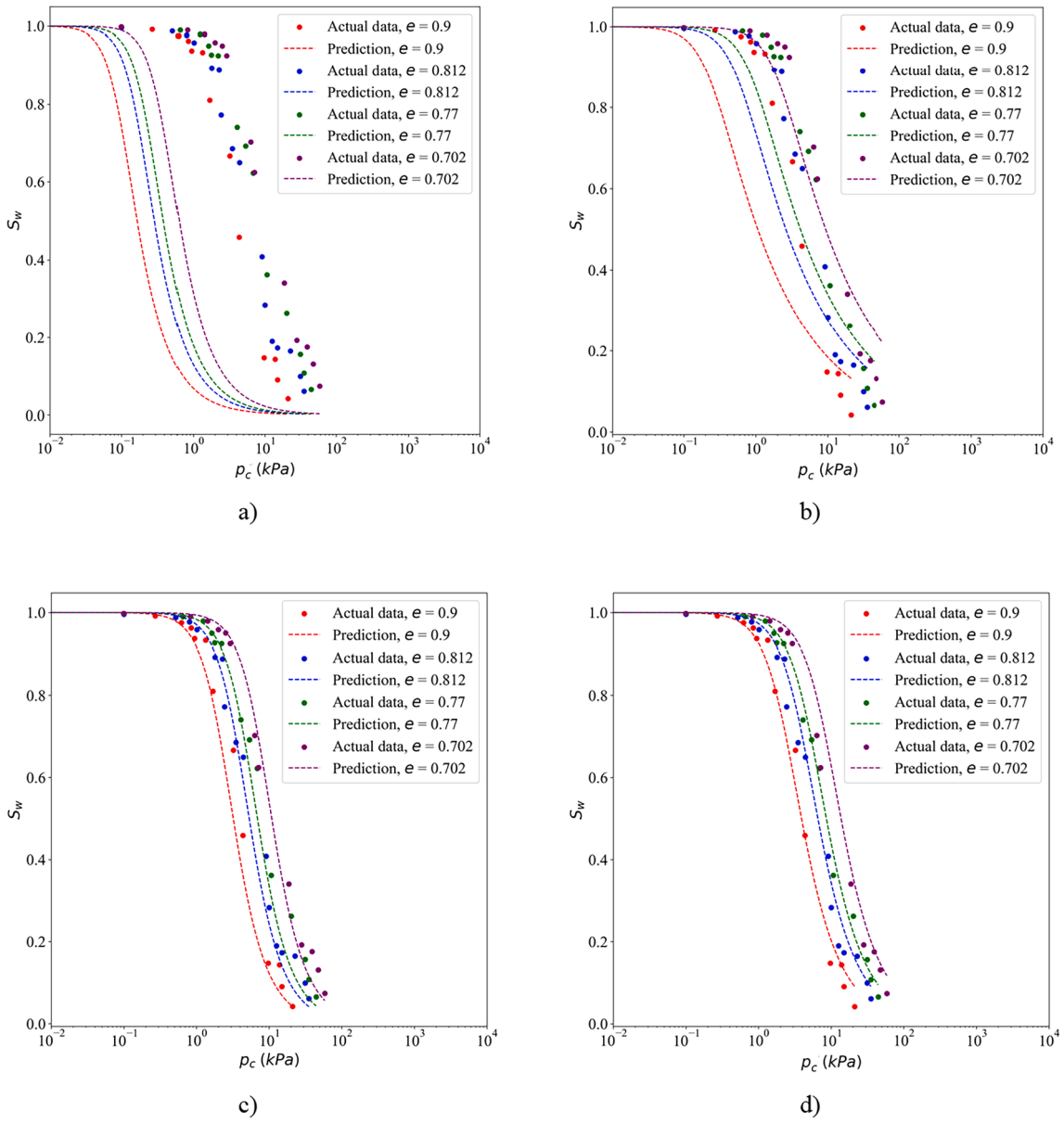


Fig. 5. The results from a sensitivity analysis on four distinct cases with varying numbers of trials (N_T): 5,000 (Panel a), 25,000 (Panel b), 40,000 (Panel c), and 50,000 (Panel d).

Our study specifically focuses on experimental data derived from Toyoura sand (Verdugo and Ishihara, 1996), examining two distinct scenarios: triaxial monotonic shearing under undrained and drained conditions, both following isotropic loading. In these experiments, the state of the material in such tests is characterized by a combination of axial strain, void ratio, mean effective stress (p'), deviatoric stress (q), and their initial mean effective stress and void ratio (p'_0 and e_0), which are crucial for determining the material's initial position relative to the critical state line. These initial conditions also affect the model predictions through the state parameter due to its role in flow rule and dilatancy laws as outlined in Equations 39 and 41. The tests' initial conditions are summarized in Table 4.

In the first series of simulations, our objective is to obtain the values of n^b, h_0, c_h, n^d, A_0, that optimize the fit, as judged by the lowest adjusted measure of relative error. It is also presumed that the critical state and elasticity parameters are derived directly from applicable laboratory tests and are known values as detailed in Table 5.

This study delineates the parameter sets utilized for model adjustment and initial testing conditions. The base parameters in set **A** comprise $a_1 = n_b$, $a_2 = h_0$, $a_3 = c_h$, $a_4 = n^d$, and $a_5 = A_0$, reflecting key properties and coefficients relevant to the constitutive model under examination. Set **G** encapsulates the initial test conditions, informed by data summarized in Table 4, and includes $g_1 = p'_0$, $g_2 = e_0$, $g_3 = \mathrm{I}_U$, and $g_4 = 1 - g_3$, which account for initial effective stress, initial void ratio, and a binary indicator for drainage conditions, respectively. It should be noted that the definitions of g_3 and g_4 ensure that the test conditions will be integrated into the ANN model as categorical variables, utilizing one-hot encoding.

The empirical parameter bounds for these parameters are assumed as follows:

- A_0 ranges from 0.05 to 1.5.
- h_0 is bounded between 1 and 30.
- n^d and n_b are both constrained within the 0.05 to 3.0 range.
- p'_0 spans from 20 to 3500 kPa.
- e_0 is set between 0.6 and 1.0.
- c_h bound is set between 0.05 and 1.0.

Table 3
Summary of the model parameters and equations.

Type	Equation	variables	Parameter
Elasticity	**Bulk Modulus**: $K = K_0 p_{atm} \frac{1+e}{e}\left(\frac{p'}{p_{atm}}\right)^{\frac{2}{3}}$ (32) **Shear Modulus**: $G =$ $G_0 p_{atm} \frac{(2.97-e)^2}{1+e}\left(\frac{p'}{p_{atm}}\right)^{\frac{1}{2}}$ (33)	e: void ratio, p': the mean effective stress, p_{atm}: atmospheric pressure –	K_0 G_0
Critical State	**Critical State Line in the $e-p'$ space**: $\ln e_c = \ln N_c - \lambda \ln(p'_c + \alpha_{CSL})$ (34) **Critical State Line in the $q-p'$ space**: $q = Mp'$ (35) **with**: $M = M_c g(\theta_l, \alpha')$, (36) $g(\theta_l, \alpha') = \left(\frac{2\alpha'^4}{1+\alpha'^4-(1-\alpha'^4)\cos 3\theta_l}\right)^{\frac{1}{4}}$, (37) $\alpha' = \frac{M_e}{M_c}$(38)	e_c: void ratio on the critical state line, p'_c: the mean effective stress on the critical state line. q: the deviatoric stress. M: the critical state slope with the corresponding values of M_e and M_c in triaxial extension and compression, respectively. Also ,θ_l: the Lode angle, α': the ratio between the critical state slope in triaxial extension and compression	N_c, λ, and α_{CSL} M_c, M_e
Kinematic hardening	**The evolution of hardening in triaxial condition**: $\frac{\partial \alpha_k}{\partial \varepsilon_q^p} = h(M\exp(\langle -n^b\psi\rangle) - m_{iso} - \eta_\sigma)$(39) **with**: $h = h_0 G_0 (1 - c_h e)\left(\frac{p'}{p_{atm}}\right)^{-\frac{1}{2}} \frac{1}{\lvert \alpha_k - a_k^{in}\rvert}$(40)	ε_q^p: the plastic deviatoric strain, α_k: the kinematic hardening parameter, m_{iso}: a sufficiently small parameter set to 0.05 M_c, η_σ: stress ratio, ψ: the state parameter by Been and Jefferies (1985) a_k^{in}: term storing the kinematic hardening parameter upon occurrence of stress reversal in cyclic loads	n^b h_0, c_h
Dilatancy	**Flow rule**: $LA_d(M\exp(n^d\psi) - m_{iso} - \eta_\sigma)$ (41) **Fabric effect on dilatancy parameter**: $A_d = A_0(1 + \langle zL\rangle)$(42) **Fabric evolution**: $dz = -c_z\langle -d\varepsilon_v^p\rangle(z_{max}L + z)$(43)	$L = 1$ or -1 when $\eta_\sigma - \alpha_k \geq 0$ and $\eta_\sigma - \alpha_k < 0$, respectively. A_d: Dilatancy parameter z: the fabric-dilatancy parameter ε_v^p: the plastic volumetric strain, $\langle\rangle$ denotes the MacCauley brackets	n^d A_0 c_z, z_{max}

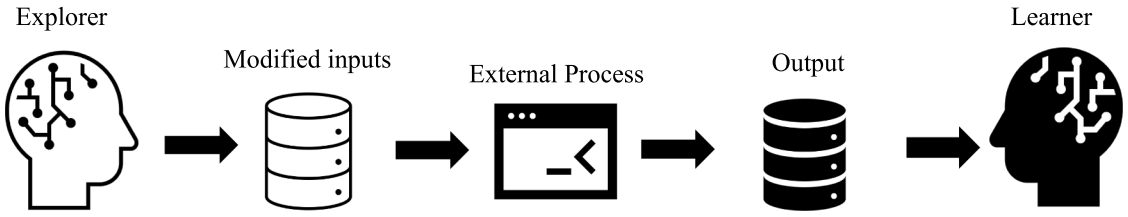


Fig. 6. Schematic representation of the self-play mechanism in the presence of an external process.

Table 4
Summary of the test initial conditions.

Condition	Name	p'_0(kPa)	e_0
Undrained	U-p'100-e0.735	100	0.735
Undrained	U-p'1000-e0.735	1000	0.735
Undrained	U-p'2000-e0.735	2000	0.735
Undrained	U-p'3000-e0.735	3000	0.735
Undrained	U-p'100-e0.833	100	0.833
Undrained	U-p'1000-e0.833	1000	0.833
Undrained	U-p'2000-e0.833	2000	0.833
Undrained	U-p'3000-e0.833	3000	0.833
Undrained	U-p'100-e0.907	100	0.907
Undrained	U-p'1000-e0.907	1000	0.907
Undrained	U-p'2000-e0.907	2000	0.907
Drained	D-p'100-e0.831	100	0.831
Drained	D-p'100-e0.917	100	0.917
Drained	D-p'100-e0.996	100	0.996
Drained	D-p'500-e0.81	500	0.81
Drained	D-p'500-e0.886	500	0.886
Drained	D-p'500-e0.96	500	0.96

Table 5
Parameters that are excluded from automatic calibration.

Symbol	Value	Unit
K_0	150	–
G_0	125	–
λ	0.37	–
α_{csl}	3370	kPa
N_c	18.7	–
M_c	1.25	–
M_e	0.89	–

In addition, I_U is defined as a binary parameter, with 1 for undrained and 0 for drained conditions.

Parameters are sampled using a uniform distribution within their defined ranges, except for I_U, which is determined through a binary random choice. The base parameters and initial conditions that will be subjected to adjustment include $A_0 = 1, h_0 = 1, n^d = 1, n_b = 1, c_h = 1$, $p'_0 = 1000kPa$, $e_0 = 0.907$, and $I_U = 1$.

The self-play process is executed by establishing the space of interest for axial strain, ε_a, within the range [0, 0.3]. The explorer agent generates 10,000 random modifications to the selected parameters and initial conditions within the permissible ranges. The process of running 10,000 simulations on a MacBook Pro equipped with an Apple M2 Pro chip and 16 GB of memory, running macOS Sonoma version 14.1 took 865.22 s.

Considering the data at hand, which encompasses both drained and undrained conditions, $\mathscr{D}$ is defined by:

$$\mathscr{D} = \{(\Delta q_i, \Delta p^{'}_i, \Delta e_i) | i = 1, 2, \cdots, N_T\} \tag{44}$$

Furthermore, $\mathscr{I}$ encompasses the current and initial mean effective stresses, deviatoric stress, and void ratio, in addition to the test condition $\mathscr{C}$ (whether drained or undrained).

$$\mathscr{I} = \{(\mathscr{T}, \mathscr{C})\} \tag{45}$$

with

$$\mathscr{T} = \left\{\left(\varepsilon_{a_i}, p^{'}_i, q_i, p^{'}_0 i, q_{0i}, e_i, e_{0i}\right) | i = 1, 2, \cdots, N_T\right\} \tag{46}$$

and

$$\mathscr{C} = \left\{\left(I_{U_i}, 1 - I_{U_i}\right) | i = 1, 2, \cdots, N_T\right\} \tag{47}$$

The definitions of **X**, **Y** and the overarching concept of the surrogate deep learning model, will be carried over unchanged from prior discussions. However, the architecture of the deep learning model has undergone modifications. The revised structure now comprises an input layer with 10 neurons, reflecting the dimensionality of **X**. This is followed by a sophisticated configuration of five hidden layers, each equipped with 256 neurons, designed to capture complex relationships and patterns within the data. The architecture results in an output layer consisting of 5 neurons. The process of training this model based on the data took 7.69 s.

In MCTS algorithm, the error assessment is defined by calculating the adjusted measure of relative errors in the following quantities of interest:

$$\mathbf{Q} = \{q, p', e\} \tag{48}$$

We conducted four distinct series of analyses by setting the action space size, N_{action}, to 100, 300, 500, 1000, and 2000, while keeping the ratio of N_{MCTS} to N_{action} constant at 1.0. The outcomes, including the adjusted measure of relative error (E_Q), computational time, and the parameters obtained, are detailed in Table 6. Our results indicate a trend where E_Q consistently diminishes as both N_{action} and N_{MCTS} increase, albeit with a corresponding rise in computational time.

Fig. 7 visualizes the progression of E_Q relative to N_{action}. In this figure, we also compare the E_Q values achieved through manual calibration, as documented by Dafalias and Taiebat (2016), Taiebat and Dafalias (2008), and Ghorbani and Airey (2021), alongside their respective E_Q values. Remarkably, the algorithm requires an action space size of

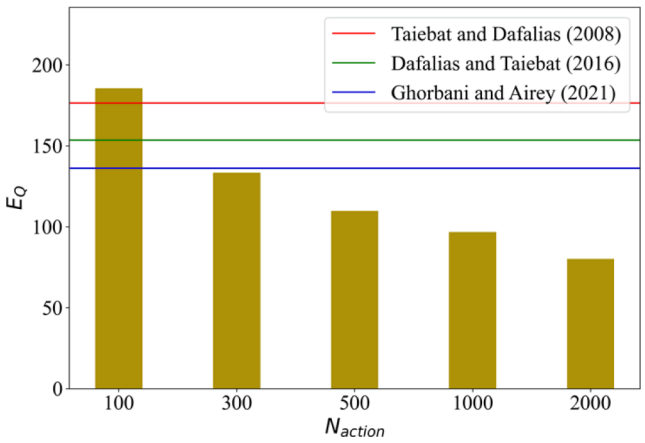


Fig. 7. Changes in E_Q with increasing N_{action} compared to manually calibrated parameters.

$N_{action} = 300$ and roughly 221 s to attain an accuracy level parallel to that of manual efforts by human experts. Beyond this point, a significant decline in E_Q is observed with increasing N_{action}, though it comes at the cost of heightened computational demands.

Fig. 8, demonstrates several examples of the algorithm's progressive refinements in predictions in the analysis with $N_{action} = 2000$.

As noted by Machaček et al. (2022) achieving an absolute fit to a set of experimental data is generally not possible, due to the inherent imperfections of the model. Additionally, the heuristic nature of the proposed algorithm and the presence of noise in experimental data can impede a perfect match with data. Therefore, it is essential to adopt a pragmatic approach in the calibration process, focusing on achieving a reasonably good fit rather than perfection. This approach should be reinforced by defining clear criteria for a good fit, which helps reduce the risk of subjectivity in calibration tasks.

The second critical point to address is the potential for non-uniqueness of parameter estimation, an issue that generally affects manual calibrations. This is illustrated by the various parameter sets derived from manual calibrations, as listed in Table 6. This issue can pose particular challenges for parameters such as A_0 and h_0, and c_h which typically lack direct calibration methods and thus rely on trial-and-error approaches (Dafalias and Manzari, 2004). This inherent variability is exacerbated by the fact that even parameters with direct calibration methods can yield different calibrated values when new or different sets of experimental data are introduced into the calibration process. This highlights the dynamic nature of calibration where newly acquired data can alter previously established parameter values, emphasizing the complexities involved in achieving stable, efficient, and consistent calibration results through manual efforts.

Nonetheless, the proposed approach addresses these challenges by: 1) introducing adjusted relative error metrics, which reduce subjectivity and provide a more objective assessment of the overall fit, and 2)

Table 6
Effect of N_{action} on analysis outcomes and comparison with manually calibrated parameters.

N_{action}	Stage of the Analysis	A_0	h_0	n_b	c_h	n^d	E_Q	CPU Time(s)	Source
100	Predicted by MCTS	0.25	9.07	1.26	0.69	2.57	185.53	79.71	–
300	Predicted by MCTS	0.52	10.74	1.54	0.909	1.95	133.4	221.74	–
500	Predicted by MCTS	0.36	11.93	2.3	0.98	1.46	109.69	362.51	–
1000	Predicted by MCTS	0.22	8.93	2.51	0.999	2.36	96.71	752.24	–
2000	Predicted by MCTS	0.28	9.53	1.59	0.995	2.85	80.02	1498.59	–
100, 300.500, 1000, and 2000	Predicted by base parameters	1	1	1	1	1	418.57	–	–
100, 300.500, 1000, and 2000	Predicted by ANN	0.82	16.09	1.50	0.59	1.59	211.22	–	–
–	–	0.4	36.96*	1.25	0.987	2.1	176.39	–	(Taiebat and Dafalias, 2008)
–	–	0.4	12	1.25	0.968	2.1	136.12		Ghorbani and Airey (2021)
–	–	0.704	15	1.25	0.987	2.1	153.43	–	(Dafalias and Taiebat, 2016)

* The Equation of hardening used by the authors is different than Equation (39) used by other cases presented in the table.

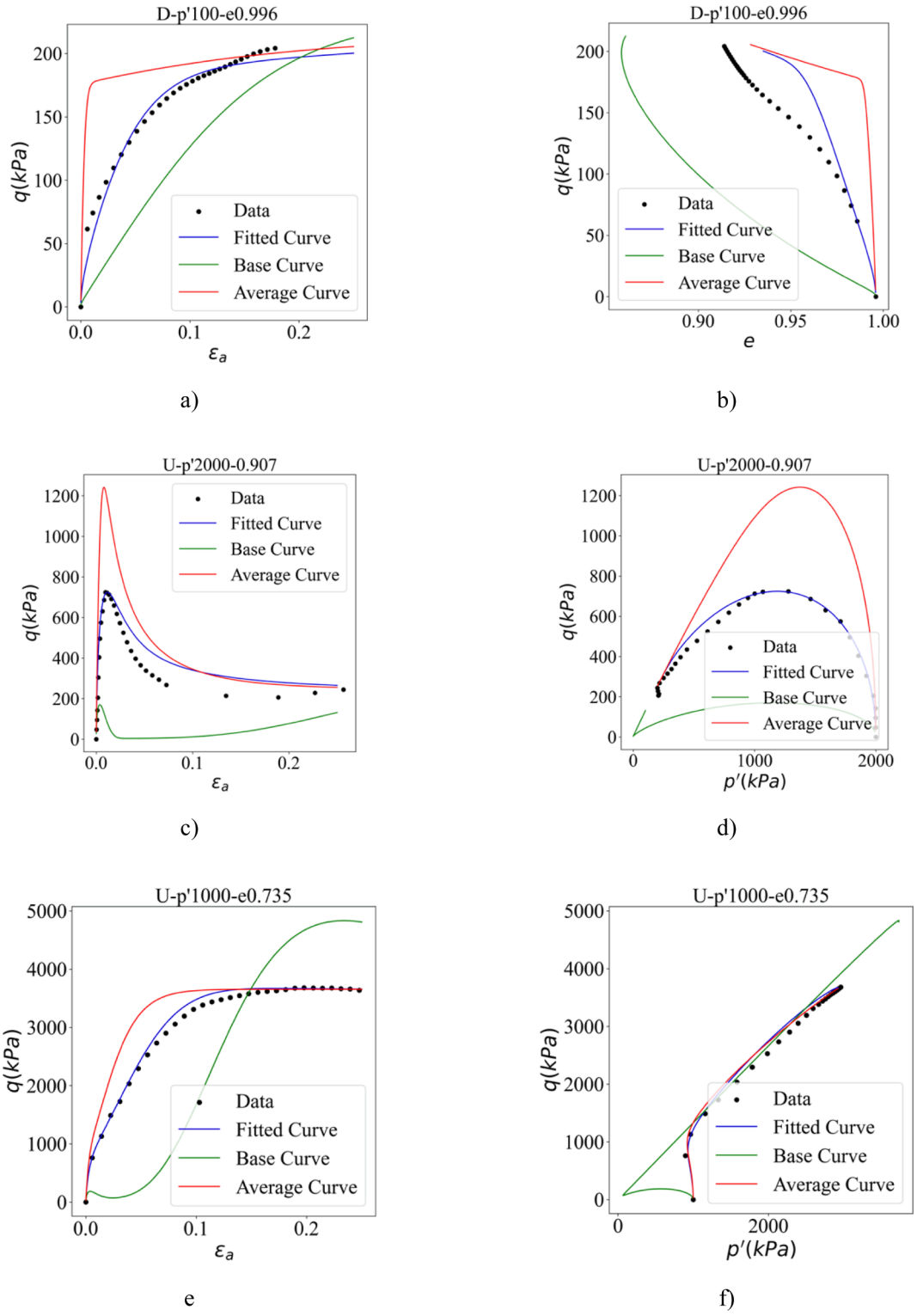


Fig. 8. The framework's ability to adeptly fit the constitutive model to experimental data through different stages: initial predictions using base parameters (denoted as "Base Curve" in the legends), subsequent refinements incorporating average adjustments $\Delta \boldsymbol{A}_{ave}$ (denoted as "Average Curve" in the legends) and after the conclusion of the game (denoted as "Fitted Curve" in the legends). Panels a) and b) display the model's predictions for test D-p'100-e0.996 in $q-e$ and $q-\varepsilon_a$ space, respectively. Panels c) and d) showing similar predictions associated with tests U-p'2000-e0.907 in $q-\varepsilon_a$ and $q-p'$ and spaces, respectively. Panels e) and f) showing similar predictions associated with tests U-p'1000-e0.735 in $q-\varepsilon_a$ and $q-p'$ and spaces, respectively. All analyses are performed by setting N_{action} to 2000.

developing an algorithm that automates the calibration process, enabling dynamic adjustments to new data sets without the need for retraining or repeated self-play.

Additionally, non-uniqueness in parameter identification can occur when the dataset lacks sufficient complexity and features to accurately determine the targeted parameters. Under these conditions, the constitutive model itself might permit non-uniqueness, where different combinations of parameters yield relatively similar optimization costs. For

example, if parameters to be calibrated are exclusively associated with cyclic behavior, the optimization algorithm may indicate their non-uniqueness when provided with a dataset limited to monotonic data. This challenge is common in all optimization and machine learning problems when parameters fall outside the scope of the available data.

Non-uniqueness may also arise from misrepresentation of physical phenomena in certain aspects of phenomenological constitutive models. In scenarios where a wide range of parameter choices does not significantly improve model predictions, the optimization algorithm may struggle to find a unique solution due to the multitude of possible solutions.

This complexity, stemming from various factors, generally cannot and should not be addressed solely by the optimization algorithm. Unless the optimization algorithm is specifically designed for data augmentation or similar approaches (which are not typically recommended in geomechanics due to the scarcity of data compared to fields like health or information technology), it is prudent to resolve this issue through other means. Ideally, this issue should be tackled by enriching the data through conducting additional relevant experiments or enhancing the constitutive model formulations. Enhancements can include improving the underlying physics, simplifying the model for specific applications, or assuming constant default values for parameters less relevant to the available data. Therefore, addressing non-uniqueness requires a comprehensive approach that includes data enrichment and model enhancement, beyond merely relying on the optimization process.

When selecting parameter ranges, choosing noninformative ranges (i.e., with broad bounds) demands more trials during self-play, requires a more complex deep surrogate model structure, and increases computational efforts by MCTS. This is due to increased values in the optimization parameters such as N_T, N_{action}, N_{MCTS} and potentially n_{sc}, all of which contribute to higher computational resource consumption and extended processing time. Consequently, the user's experience in determining suitable bounds can significantly impact the efficiency of the optimization process.

There are several practical approaches for determining suitable parameter bounds for the model in numerical simulations in geomechanics, which can be applied in the context of this algorithm:

Literature-Based Information: Using available information in the literature, parameters for soils similar to the soil at hand can be used as initial guesses. The domain can then be defined by these values plus a margin based on collected data. This approach was selected in this study because both the SANISAND family models and the selected SWRC model are well-documented.

Empirical Equations: Utilizing empirical or physics-based equations that relate fundamental soil characteristics, such as grading and particle shape, to the model's parameters (e.g., SWRC and constitutive parameters) (Alaei et al., 2022; Pande et al., 2020).

Parameter Reduction: Reducing the parameters involved in calibration by directly estimating them from experimental data where possible or by assuming default values if they are expected to have negligible impact on the analysis. For example, default values for parameters associated with cyclic data can be used when the application primarily focuses on monotonic behavior. This approach is also pursued in this paper specifically parameters z_{max} and c_z are set to default values while the critical state and elasticity parameters were directly obtained from the data.

Future applications can focus on creating a database to improve optimization efficiency by keeping parameters within a reasonable and plausible range.

3.3. Application to geotechnical inverse problems

This section discusses application of the introduced method to inverse problems, which are widely encountered in geotechnical applications such as compacted ground assessment. In this class of problems, the objective is to deduce specific ground properties from the observed test data. Solving this class of problems can be challenging, especially when determining properties of unsaturated soils. This challenge stems from the interplay of various parameters, such as moisture content, density, and suction as well as the characteristics of interaction between the soil and the testing device, on test outcomes (Ghorbani et al., 2024). These inverse problems are generally beyond the direct scope of conventional methods like the finite element method and require tailored approaches.

We specifically discuss extraction of the parameter of a nonlinear elastic model, utilizing the Light Weight Deflectometer (LWD) data. LWD, a non-destructive technique (see Fig. 9), assesses soil properties by recording deformations resulting from an impulse load, achieved by dropping a 10 kg mass. The LWD device yields a pseudo-Young's modulus E_{LWD} under the assumptions of homogeneity and linear elastic behavior within a framework that considers the soil as isotropic and single-phase medium, using the following equation (Schwartz et al., 2017):

$$E_{LWD} = \varsigma \frac{(1 - \nu_{LWD}{}^2)}{\pi R_{LWD}} K_d \tag{49}$$

where the dynamic hardness, K_d is the peak load divided by the peak deflection, both measured by the device during the tests.

The comprehensive methodologies utilized in the LWD tests, the results of which are summarized in Table 7, serve as the basis for evaluating the developed model in the forthcoming analyses. While the precise measurement techniques and procedural details are exhaustively described by Ghorbani et al. (2024), and hence not repeated here, it is essential to highlight key parameters employed across these tests.

The stress distribution factor, ς and Poisson's ratio, ν_{LWD}, are set to the device's default values of 2.0 and 0.35, respectively. The radius of the plate in contact with the soil, R_{LWD}, is 150 mm. The dynamic load peaks at approximately 6.5 kN, with the impulse duration noted at 0.028 s, as elaborated by a series of experimental results reported by Ghorbani et al. (2024). These tests were performed within a custom-made container filled with a silty sand called SPARC sand (Dutta and Kodikara, 2022).

In their investigation into the effects of dry density and moisture content on the observed LWD moduli, (Ghorbani et al., 2024) tackled this problem as a forward problem by employing a fully coupled finite element model (e.g., Schrefler and Scotta, 2001, Airey and Ghorbani, 2021, Khoei and Saeedmonir, 2021) alongside a mortar-based contact algorithm. In their method, they also utilized the set of soil parameters, which are summarized in Table 8, derived from laboratory tests to evaluate accuracy of a finite element model of LWD test on SPARC Sand in reflecting observed moduli.

In this forward modeling scenario, specific soil properties and initial conditions, such as density and moisture content, were inputs into the finite element model, generating LWD moduli outcomes based on the predicted deflections. These outcomes were then compared with experimental moduli for validation purposes. Their findings highlighted the critical role of effective stress in the accuracy of the analyses. Also, the authors showed that, particularly under conditions distant from saturation (such as those listed in Table 7, the implementation of a nonlinear elastic model in compacted SPARC Sand tended to yield reasonably accurate results. However, as conditions approached saturation, where plastic deformations predominate, the accuracy in predicting LWD moduli diminished.

This paper frames the same problem as an inverse problem with the core objective of deducing the soil's bulk modulus, now considered an undetermined parameter, using the presented AI-driven algorithm. The finite element model is the same as the one presented by Ghorbani et al., (2024) and incorporates the mortar-type contact algorithm developed

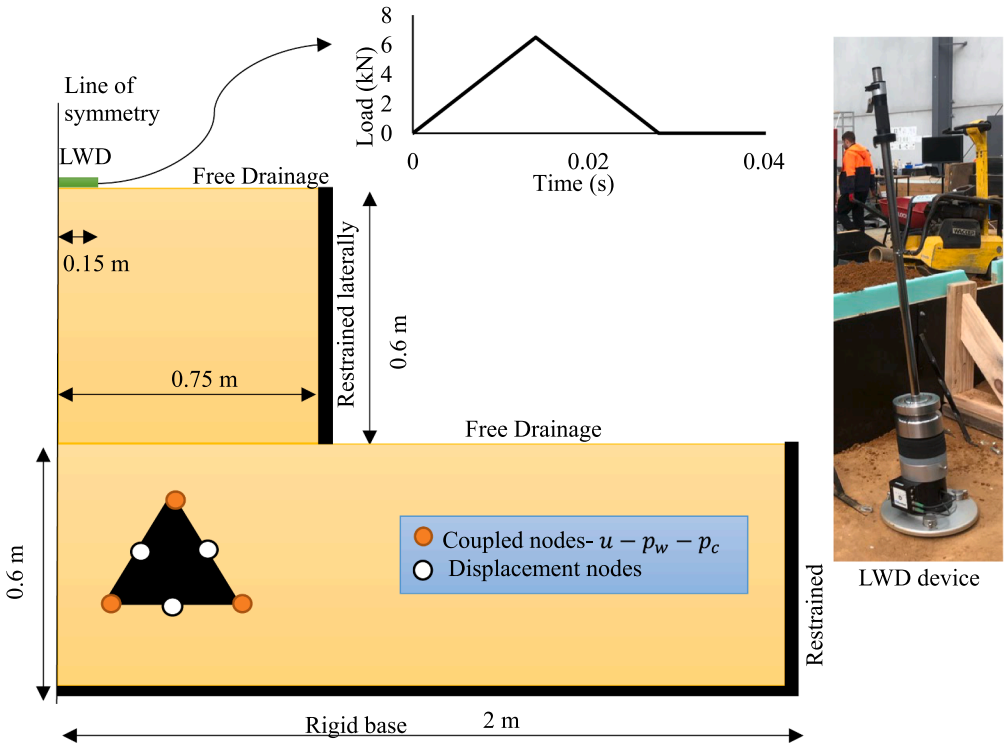


Fig. 9. The idealised axisymmetric model and the applied load along with the LWD device used in the study by Ghorbani et al. (2024).

Table 7
The LWD values in the tests.

Tests	Measured LWD (MPa)	Measured gravimetric water content (%)	Void ratio	Degree of saturation
Test w1d1	100.62	4.1	0.63	0.18
Test w1d2	108.61	4.6	0.60	0.21
Test w2d1	68.86	6.6	0.54	0.33
Test w2d2	192.62	6.4	0.42	0.41
Test w3d1	73.86	9.1	0.38	0.66
Test w3d2	111.47	8.5	0.33	0.69

by Ghorbani et al., (2021b), to accurately simulate the contact between the LWD plate and unsaturated soil.

A summary of key governing equations in the finite element model is given in this study. Upon linearizing the additional energy due to frictionless contact in the governing equations, the finite element discretization of the fully coupled equations is formulated as follows:

$$\mathbf{M_u}\ddot{\mathbf{U}} + \mathbf{C}\dot{\mathbf{U}} + \mathbf{K}\mathbf{U} + \bigcup_{i=1}^{n_s}\mathbf{K}_{NC_i} - \mathbf{Q_w}\mathbf{P_w} - \mathbf{Q_c^*}\mathbf{P_c} = \mathbf{F_u} + \bigcup_{i=1}^{n_s}\mathbf{F}_{NC_i} \tag{50}$$

$$\mathbf{M_w}\ddot{\mathbf{U}} + \mathbf{Q_w^T}\dot{\mathbf{U}} + \mathbf{C_{ww}}\dot{\mathbf{P}}_\mathbf{w} + \mathbf{C_{wc}}\dot{\mathbf{P}}_\mathbf{c} + \mathbf{H_{ww}}\mathbf{P_w} + \mathbf{H_{wc}}\mathbf{P_c} = \mathbf{F_w} \tag{51}$$

$$\mathbf{M_c}\ddot{\mathbf{U}} + \mathbf{R_c^T}\dot{\mathbf{U}} + \mathbf{C_{cw}}\dot{\mathbf{P}}_\mathbf{w} + \mathbf{C_{cc}}\dot{\mathbf{P}}_\mathbf{c} + \mathbf{H_{wc}^T}\mathbf{P_w} + \mathbf{H_{cc}}\mathbf{P_c} = \mathbf{F_c} \tag{52}$$

where n_s represents the number of active contact segments. The primary variables in these equations are the displacement ($\mathbf{U}$), pore water pressure ($\mathbf{P_w}$), and suction ($\mathbf{P_c}$).

Force vectors and matrices, are symbolized as F, M, Q, H, and C. The subscript NC refers to normal contact, while the subscripts u, w, and c indicate elements related to displacement, pore water pressure, and suction degrees of freedom, respectively. Comprehensive definitions of all vectors and matrices, along with the contact-related terms $\mathbf{K}_{NC_i}$ and $\mathbf{F}_{NC_i}$ are found in (Ghorbani et al., 2021b, Ghorbani and Kodikara, 2023). Furthermore, the SWRC equation is the same as that outlined in Equation (26).

A schematic representation of geometry, boundary condition, and the applied impulse load are shown in Fig. 9. The geostatic stress is established initially and in the subsequent step, the dynamic plate-soil interaction is simulated using 1,000 time steps of size 0.00003 s.

Our objective is to infer K_0 in Equation 32 using the set of measurements and initial conditions provided in Table 7. The base parameters in set **A** comprises only $a_1 = K_0$. Set **G** encapsulates the initial test conditions, informed by data summarized in Table 7, and includes $g_1 = e_0$ and $g_2 = w_0$, which account for initial void ratio, and moisture content, respectively.

The parameter bounds are as follows:

- K_0 ranges from 125 to 300.
- e_0 is set between 0.25 and 0.65.
- w_0 bound is set based on the e_0 range and in conjunction with the following range for the degree of saturation variation: 0.05 and 1.0.

The explorer agent generates 1,000 random modifications across the selected parameters and initial conditions within the permissible ranges. The process of automatic execution of 1,000 simulations on a MacBook Pro equipped with an Apple M2 Pro chip and 16 GB of memory, running macOS Sonoma version 14.1 took 3760.32 s. The base K_0 parameter is set to 125.

We define $\mathscr{D}$ as follows:

Table 8
The constitutive and general material parameters (Ghorbani et al., 2024).

Type	Description	Symbol	Value	Unit
Elastic constitutive model parameters	Reference shear modulus	G_0	125	–
	Reference bulk modulus*	K_0*	150*	–
General material parameters	Density of solid particles	ρ_s	2700	kg m^{-3}
	Density of water	ρ_w	997	kg m^{-3}
	Density of air	ρ_a	1.1	kg m^{-3}
	Bulk modulus of water	K_w	2.25×10^6	kPa
	Bulk modulus of air	K_a	1.01×10^2	kPa
	Intrinsic permeability in the saturated state	k	1×10^{-10}	m^2
	Viscosity of water	η_w	1.0×10^{-3}	Ns m^{-2}
	Viscosity of air	η_a	1.8×10^{-5}	Ns m^{-2}
SWRC parameters	Controls the slope of the main drying curve	n^d	0.28	–
	Controls the slope of the main drying curve	m^d	0.98	–
	Air-entry value of the main drying curve at a reference void ratio of 1	P_d	0.012	kPa
	Controls the contribution of void ratio on the air-entry value	Ω'	10.6	–

* This parameter is treated as unknown in this study.

$$\mathscr{D} = \{(\Delta E_{LWD\,i}) | i = 1, 2, \cdots, N_T\} \tag{53}$$

Furthermore, $\mathscr{I}$ includes E_{LWD}, the initial void ratio and moisture content as follows:

$$\mathscr{I} = \left\{ \left(E_{LWD_i}, e_{0i}, w_{0i}\right) | i = 1, 2, \cdots, N_T \right\} \tag{54}$$

The definitions of **X**, **Y** and the surrogate deep learning model remain as previously discussed, the architecture of the deep learning model itself has been refined. The updated model architecture is structured as follows:

- An input layer featuring four neurons, aligning with the dimensionality of **X**, which typically represents the input features or conditions under which the soil's response is being modeled.
- Three hidden layers, each outfitted with 64 neurons, designed to capture and process the nonlinear relationships between the inputs and the desired output.
- A concluding output layer with a single neuron, responsible for generating the predicted value of ΔK_0.

Table 9 provides an insightful overview of the outcomes from a sequence of analyses, differentiated by varying N_{action} sizes: 10, 50, 100, 300, and 500. Consistently across these scenarios, the ratio of N_{MCTS} to N_{action} is maintained at 1.0. This analysis framework demonstrates a notable increase in computational time, particularly as the size of the action increases, with the 500-action analysis resulting in approximately 3 h of computational effort. This escalation in time is anticipated, especially when compared to the computational demands of previously tackled constitutive model problems. Nevertheless, a consistent trend emerges from this data: as N_{action} enlarges, there is a systematic closeness to the K_0 value given in Table 8 and reduction in E_Q and, echoing findings from earlier experiments.

Table 9
The selected values of N_{action} and corresponding predicted K_0, E_Q, and time of the analysis.

N_{action}	K_0	E_Q	Time (s)
10	137.6	1.38	269.19
50	143.2	1.16	1241.21
100	145.4	1.07	2355.38
300	146.9	1.01	7372.93
500	148.0	0.97	10992.11

4. Conclusions

In this study, we have developed AlphaGeo, an innovative AlphaZero-inspired AI framework tailored for the autonomous identification of parameters in sophisticated geotechnical models from experimental and sensing data. Inspired by DeepMind's AlphaZero, this approach reconceptualizes parameter identification problems as strategic games. It pioneers a self-learning mechanism that operates independently of human-derived insights, except for the initial definition of the game's rules.

A pivotal advancement of our method is its capacity to circumvent the limitations inherent in both gradient-based and gradient-free optimization techniques commonly encountered in this domain. Notably, it addresses the issue of retaining learned experiences; once self-play has been conducted and learning completed for a set of parameters, any subsequent data can be directly applied to the pre-trained model for further parameter identification. This feature ensures a swift and effective adaptation to new data and can aid employing more precise geotechnical models for interpreting sensing and experimental data.

The effectiveness of our approach is demonstrated across a diverse range of geotechnical applications, including the calibration of advanced constitutive models and complex finite element simulations that simulate the interaction between mechanical sensing devices and soil. By integrating the predictive prowess of deep learning with the strategic refinement offered by MCTS, this dual-phase strategy guarantees robust calibration, even in instances where initial predictions by the deep surrogate model may falter.

The successful implementation of our AlphaZero-inspired strategy signifies a considerable advancement towards unlocking the full potential of AI in improving the applicability of advanced geotechnical model, particularly in the context of interpreting sensing data.

The algorithm presented in this study is particularly beneficial for scenarios where exact solutions are unattainable due to inherent model imperfections and noisy data, which are common challenges in geomechanics when dealing with real-time data. In these situations, the objective is often to perform repetitive tasks, such as finding an approximate solution for the same model under varied initial conditions and data input. This requirement, along with the algorithm's capacity to retain learned experiences and reduce human involvement during the process, renders it highly suitable for practical geotechnical applications, particularly when rough estimations about the parameter ranges can be obtained.

Future endeavors could concentrate on enhancing the computational efficiency of the algorithm, especially in tackling complex boundary value problems, and expanding the range of application.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT for proofreading. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Javad Ghorbani: Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Sougol Aghdasi:** Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Majidreza Nazem:** Writing – review & editing, Validation. **John S McCartney:** Writing – review & editing, Validation. **Jaynatha Kodikara:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This research work is part of a research project (Project No IH18.03.1) sponsored by the SPARC Hub at the Department of Civil Engineering, Monash University funded by the Australian Research Council (ARC) Industrial Transformation Research Hub (ITRH) Scheme (Project ID: IH180100010). The financial and in-kind support of EIC Activities of CIMIC Group, Austroads, and Monash University is gratefully acknowledged. Also, the financial support of ARC is highly acknowledged.

References

Airey, D.W., Ghorbani, J., 2021. Analysis of unsaturated soil columns with application to bulk cargo liquefaction in ships. Comput. Geotech. 140, 104402.
Alaei, E., Marks, B., Einav, I., 2022. A five-parameter hydrodynamic-plastic model for crushable sand. Int. J. Solids Struct. 254, 111914.
Auer, P., Cesa-Bianchi, N., Fischer, P., 2002. Finite-time analysis of the multiarmed bandit problem. Mach. Learn. 47, 235–256.
Been, K., Jefferies, M.G., 1985. A state parameter for sands. Géotechnique 35, 99–112.
Chen, L., Ghorbani, J., Kodikara, J., 2024. Modelling of shakedown, ratcheting and liquefaction of saturated granular soils using kinematic hardening with memory surface. Comput. Geotech. 165, 105830.
Dafalias, Y.F., Manzari, M.T., 2004. Simple plasticity sand model accounting for fabric change effects. J. Eng. Mech. 130, 622–634.
Dafalias, Y.F., Taiebat, M., 2016. SANISAND-Z: zero elastic range sand plasticity model. Géotechnique 66, 999–1013.
Dutta, T.T., Kodikara, J., 2022. Evaluation of unbound/subgrade material rutting and resilient behaviour based on initial density and saturation degree. Transp. Geotech. 35, 100782.
Gallipoli, D., Gens, A., Sharma, R., Vaunat, J., 2003a. An elasto-plastic model for unsaturated soil incorporating the effects of suction and degree of saturation on mechanical behaviour. Géotechnique. 53, 123–136.
Gallipoli, D., Wheeler, S., Karstunen, M., 2003b. Modelling the variation of degree of saturation in a deformable unsaturated soil. Géotechnique 53, 105–112.
Ghorbani, J., Airey, D.W., 2021. Modelling stress-induced anisotropy in multi-phase granular soils. Comput. Mech. 67, 497–521.
Ghorbani, J., Airey, D.W., Carter, J.P., Nazem, M., 2021a. Unsaturated soil dynamics: finite element solution including stress-induced anisotropy. Comput. Geotech. 133, 104062.
Ghorbani, J. Airey, D.W. 2019. Some aspects of numerical modelling of hydraulic hysteresis of unsaturated soils. *Unsaturated soils: Behavior, mechanics and conditions.* Nova Science Publishers.
Ghorbani, J., Kodikara, J., 2023. Thermodynamically consistent effective stress formulation for unsaturated soils across a wide range of soil saturation. Comput. Mech. 1–18.
Ghorbani, J., Kodikara, J., 2024. Real-time inference of compacted soil properties using deflection tests: an AI-driven solution informed by unsaturated soil mechanics principles. Comput. Geotech. 173, 106543.
Ghorbani, J., Nazem, M., Kodikara, J., Wriggers, P., 2021b. Finite element solution for static and dynamic interactions of cylindrical rigid objects and unsaturated granular soils. Comput. Methods Appl. Mech. Eng. 384, 113974.
Ghorbani, J., Sountharajah, A., Dutta, T.T., Kodikara, J., 2024. Modelling rapid non-destructive test using light weight deflectometer on granular soils across different degrees of saturation. J. Rock Mech. Geotech. Eng.
Kadlíček, T., Janda, T., Šejnoha, M., Mašín, D., Najser, J., Beneš, Š., 2022. Automated calibration of advanced soil constitutive models. Part I: hypoplastic sand. *Acta Geotechnica* 1–18.
Khoei, A., Saeedmonir, S., 2021. Computational homogenization of fully coupled multiphase flow in deformable porous media. Comput. Methods Appl. Mech. Eng. 376, 113660.
Kinikles, D., Rong, W., McCartney, J., 2024. Hyperbolic hydro-mechanical model for seismic compression of unsaturated sands in the funicular regime. Comput. Geotech. 167, 106113.
Lecampion, B., Constantinescu, A., Nguyen Minh, D., 2002. Parameter identification for lined tunnels in a viscoplastic medium. Int. J. Numer. Anal. Meth. Geomech. 26, 1191–1211.
Ledesma, A., Gens, A., Alonso, E., 1996. Estimation of parameters in geotechnical backanalysis—I maximum likelihood approach. Comput. Geotech. 18, 1–27.
Levasseur, S., Malécot, Y., Boulon, M., Flavigny, E., 2008. Soil parameter identification using a genetic algorithm. Int. J. Numer. Anal. Meth. Geomech. 32, 189–213.
Li, T., Li, X., Rui, Y., Ling, J., Zhao, S., Zhu, H., 2024. Digital twin for intelligent tunnel construction. Autom. Constr. 158, 105210.
Machaček, J., Staubach, P., Tavera, C.E.G., Wichtmann, T., Zachert, H., 2022. On the automatic parameter calibration of a hypoplastic soil model. Acta Geotech. 17, 5253–5273.
Maghvan, S.V., Imam, R., McCartney, J.S., 2019. Relative density effects on the bearing capacity of unsaturated sand. Soils Found. 59, 1280–1291.
Mendez, F.J., Pasculli, A., Mendez, M.A., Sciarra, N., 2021. Calibration of a hypoplastic model using genetic algorithms. Acta Geotech. 16, 2031–2047.
Pande, G., Pietruszczak, S., Wang, M., 2020. Role of gradation curve in description of mechanical behavior of unsaturated soils. Int. J. Geomech. 20, 04019159.
Petalas, A.L., Dafalias, Y.F., Papadimitriou, A.G., 2020. SANISAND-F: Sand constitutive model with evolving fabric anisotropy. Int. J. Solids Struct. 188, 12–31.
Robson, J.D., Armstrong, D., Cordell, J., Pope, D., Flint, T.F., 2024. Calibration of constitutive models using genetic algorithms. Mech. Mater. 189, 104881.
Schrefler, B.A., Scotta, R., 2001. A fully coupled dynamic model for two-phase fluid flow in deformable porous media. Comput. Methods Appl. Mech. Eng. 190, 3223–3246.
Schwartz, C. W., Afsharikia, Z. Khosravifar, S. 2017. Standardizing lightweight deflectometer modulus measurements for compaction quality assurance. Maryland. State Highway Administration.
Seidl, D.T., Granzow, B.N., 2022. Calibration of elastoplastic constitutive model parameters from full-field data with automatic differentiation-based sensitivities. Int. J. Numer. Meth. Eng. 123, 69–100.
Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., 2017. Mastering the game of go without human knowledge. Nature 550, 354–359.
Sloan, S.W., Abbo, A.J., Sheng, D., 2001. Refined explicit integration of elastoplastic models with automatic error control. Eng. Comput. 18, 121–154.
Taiebat, M., Dafalias, Y.F., 2008. SANISAND: Simple anisotropic sand plasticity model. Int. J. Numer. Anal. Meth. Geomech. 32, 915–948.
Verdugo, R., Ishihara, K., 1996. The steady state of sandy soils. Soils Found. 36, 81–91.
Yu, S., Lei, Q., Liu, C., Zhang, N., Shan, S., Zeng, X., 2023. Application research on digital twins of urban earthquake disasters. Geomat. Nat. Haz. Risk 14, 2278274.
Zentar, R., Hicher, P.-Y., Moulin, G., 2001. Identification of soil parameters by inverse analysis. Comput. Geotech. 28, 129–144.